AdaptiveCache: Layout-Aware Context Management for Prefix-Cache-Efficient LLM Agents

Vlad Cainamisir
Harvard John A. Paulson School of Engineering and Applied Sciences
CS2640: Modern Storage Systems — Midterm Checkpoint, April 2026

Abstract

LLM agents accumulate context across hundreds of tool calls. Compaction is inevitable—but every existing approach (FIFO eviction, summarization, RL compression) leaves survivors in a configuration that makes future requests pay full recompute cost, because the prefix byte sequence has changed. We argue that compaction should be a *layout optimization problem*: choose not just what to keep but where to place survivors, so that the prefix byte sequence seen by the next request is identical to the last, and its KV states are already cached. AdaptiveCachepins stable, high-value blocks at fixed prefix positions and evicts volatile blocks from the suffix, amortizing one layout reorganization across many future cache hits.

We report two experiments with honest discussion of their limitations. A simple Anthropic-API prototype confirms that naive message-level eviction (FIFO, compaction) costs more than no eviction at all—cache invalidation outweighs token savings. A controlled vLLM experiment shows that suffix-first eviction holds prefix cache hit rate at $\sim$99% through eviction, while prefix-first eviction (FIFO) collapses to $\sim$16%, but suffers from warm GPU cache contamination. We then describe what Anthropic's production system (Claude Code) and the concurrent MEMENTO paper reveal about the space, and conclude that the heuristic scoring approach is insufficient—the real contribution should be a data-driven scorer trained on attention traces at scale.

1 Introduction

Modern LLM inference servers implement *prefix caching*: if consecutive requests share a byte-identical prefix, KV states for those tokens are reused, reducing effective compute from $O(N)$ to $O(1)$ for cached positions. For a long-running agent, prefix caching can eliminate 80–95% of prompt compute. The requirement is strict: any modification to early tokens invalidates all downstream cached KV states.

Context management is therefore a *storage layout problem*. Every agent context eventually exceeds its budget and must be compacted. The question is not just *what to keep*, but *how to arrange survivors* so that the resulting prefix byte sequence is identical to the previous request, and its KV states are already cached in the server. Compact correctly and you pay one layout reorganization amortized across many future steps. Compact naively and every subsequent step pays full recompute—making compaction self-defeating.

This is the **layout gap**: every existing system optimizes what to keep but ignores where survivors land. KV eviction methods [3,5,7,11,13] scatter survivors at arbitrary positions—a different configuration per step, no cross-step prefix reuse. Summarization systems [1, 8, 10] generate new tokens, creating a byte sequence with zero cached prefix. RL-trained approaches [9, 12, 14] require retraining and still replace the prefix at each turn.

AdaptiveCachecloses the layout gap: identify stable, high-value blocks and pin them at fixed prefix positions; evict volatile blocks from the suffix by leaving holes, never renumbering; never summarize as a primary mechanism. One layout reorganization establishes the right prefix; future steps append to the suffix and get continuous cache hits until the importance structure changes enough to warrant another reorganization.

2 Related Work

2.1 KV-Cache Eviction (Inference-Layer)

These systems reduce inference memory by evicting KV entries during generation. They share the layout gap: correct identification of important tokens, but no cross-step prefix stability.

H2O [13]. Attention weights follow a power law: a small "heavy hitter" set captures most mass. H2O maintains this set plus a recent window, achieving 1.9$\times$ throughput. AdaptiveCacheborrows the cumulative attention signal.

StreamingLLM [11]. Initial tokens receive disproportionate attention as a SoftMax dump ("attention sinks"). Keeping 4 sink tokens + a rolling window enables infinite streaming

at 22.2× speedup. This doubly motivates AdaptiveCache's stable prefix: initial tokens are both semantically load-bearing and empirically critical.

ScissorHands [7]. Proves that >95% of high-attention tokens in the first half of generation are also high-attention in the second half (*persistence of importance*). This justifies AdaptiveCache's use of importance variance as a stability signal—consistently important blocks can be predicted and pinned early.

SnapKV [5] and PyramidKV [3] extend this with per-head and per-layer selection, and Liu *et al.* [6] show empirically that content at fixed early positions is attended to far more reliably than middle-scattered content—a behavioral complement to the caching argument. All remain per-query with no cross-step stability.

2.2 Agent Context Management

Summarization-based. ReSuM [10] is a plug-and-play external summarizer (+4.5% over ReAct, training-free). SUPO [8] trains a GRPO summarization policy (+14% BrowseComp-Plus). OpenHands [1] deploys nine composable condensation strategies in production. All generate new summary tokens, invalidating the prefix cache. ReSuM training-free is AdaptiveCache's primary comparison target.

RL-trained state replacement. MEM1 [14] replaces the entire context with a fresh internal state each turn via PPO (3.5× memory reduction, 3.7× task performance). MemAgent [12] maintains a 1024-token overwrite memory via Multi-Conv DAPO, extrapolating to 3.5M tokens. Both require training; applying either training-free consistently hurts performance—a direct warning for AdaptiveCache's heuristic scoring approach.

Context Folding [9]. Trains an RL agent to branch into sub-trajectories and fold them upon completion (62% BrowseComp-Plus, 58% SWE-bench). It operates at the sub-trajectory level, not the token level, and is not prefix-cache-aware.

2.3 Summary

3 System Design

3.1 Block Segmentation and Scoring

AdaptiveCacheparses the agent's OpenAI-format message list into typed *blocks*: SYSTEM, TASK, THOUGHT, ACTION, and four observation types (OBS_FILE, OBS_SHELL, OBS_GREP, OBS_ERROR). Each block is scored on two independent axes:

Importance. A weighted sum of: (a) *structural type prior*—file reads score 0.6, shell outputs 0.3, system/task 1.0; (b) *reference count*—how many times the block's identifiers (file paths, function names) appear in subsequent thoughts; (c) *cumulative attention* (100% milestone stub, requires real weights from the model).

Stability. Whether importance is consistent over time: (a) *type stability prior*—file reads are stable (0.7), error messages volatile (0.1); (b) *importance variance*—low variance over recent steps implies the block can safely be pinned.

The combined *pin score* $p = \text{importance} \times \text{stability}$ drives zone assignment. The *eviction priority* $e = \text{importance} \times (1 - \text{stability})$ evicts hot-but-volatile blocks before cold-but-stable ones.

3.2 Three-Zone Layout and Eviction

Blocks are assigned to three zones: PIN ($p > \theta_{\text{pin}}$), MIDDLE ($p > \theta_{\text{mid}}$), and SUFFIX (everything else). Eviction fires SUFFIX-first when the total token count exceeds budget, cascading to MIDDLE if needed, and never touching PIN.

The PIN zone is reconstructed in fixed order at the prefix on every step— system prompt, task description, and high-value file reads at identical absolute positions. The byte sequence at the prefix is therefore identical across consecutive steps, enabling continuous prefix cache hits.

A layout reorganization fires every ∼15 steps (or when the top-K blocks change composition by >30%), invalidating the cache for one step and establishing a new stable layout thereafter.

3.3 Three-Tier Runtime Architecture

The system operates three tiers at different timescales:

Tier 1 — every step. Low-scoring blocks are marked for eviction and their KV entries deleted from LMCache via `POST /delete_blocks`. Because modern LLMs use post-rotated RoPE keys baked in at compute time, evicted positions become unattendable without invalidating any downstream KV state [5, 13]. No new tokens, no LLM calls, no recomputation.

Tier 2 — every ∼15 steps. When the top-K block composition changes by >30%, or accumulated holes exceed 40% of allocated positions, the layout optimizer re-sorts surviving blocks into PIN → MIDDLE → SUFFIX order. This invalidates the prefix for one step; the new layout is then stable for the next ∼15 steps.

Tier 3 — emergency only. If eviction alone cannot meet the budget (e.g., the PIN zone is overflowing), a full LLM summarization replaces accumulated context. This always invalidates the prefix; the goal is to never reach this tier.

3.4 A Critical Note on the Heuristic Scorer

The scoring framework described above is entirely heuristic. No signal in the current implementation is grounded in empirical data: the type priors (file reads score 0.6, shell outputs 0.3)

System	Training required?	Layout-aware?	Prefix-cache-aware?
FIFO / LRU	No	No	No
H2O / SnapKV / ScissorHands / PyramidKV	No	No	No
ReSuM [10]	Optional	No	No
MEM1 [14]	Yes	No	No
MemAgent [12]	Yes	No	No
Context Folding [9]	Yes	No	Partial
MEMENTO [4]	Yes	No	No†
AdaptiveCache(this work)	**No**	**Yes**	**Yes**

† MEMENTO explicitly disables prefix caching; see §4.

Table 1: AdaptiveCacheis the only system that is layout-aware and prefix-cache-aware without requiring training.

Tier	Operation	Cost	Freq.
1	KV block deletion	≈free	Every step
2	Layout reorganization	Moderate	∼15 steps
3	Summarization (fallback)	Expensive	Emergency

Table 2: Three-tier pipeline. Goal: never reach Tier 3.

are educated guesses, the reference-counting signal relies on regex identifier matching, and the stability signals are structural proxies with no validation. This scorer was produced by the AI coding assistant and, on reflection, is not a heuristic the author finds convincing.

The 0/5 SWE-bench patch submission rate confirms this directly: the scorer evicts file reads the agent needs, forces re-reads, and increases cost. The heuristics are not wrong in direction but far too coarse for the decisions that actually matter—motivating the data-driven pivot in §7.

4 Concurrent Work and Production Evidence

4.1 Claude Code Compaction [2]

Claude Code's compaction source code became publicly accessible in early 2026. It reveals a four-stage cheapest-first pipeline: **Snip** (sliding window) → **Microcompact** (tool result clearing, no LLM call) → **Context Collapse** (subproblem folding) → **Autocompact** (full LLM summarization, last resort). This is the same tier hierarchy AdaptiveCacheindependently arrived at.

The most significant finding is *cached microcompact*: when the server-side prompt cache is warm, Claude Code enqueues `cache_edits` — API instructions to surgically delete KV entries for specific old tool results *without invalidating the rest of the prefix*. This is exactly AdaptiveCache's Tier 1, running at production scale:

```
return {
  messages,  // local messages UNCHANGED
  compactionInfo: { pendingCacheEdits: {
    deletedToolIds: toolsToDelete, ... } }
}
```

Limitations relative to AdaptiveCache: targets only tool results; requires an Anthropic-internal API; no layout step. AdaptiveCachegeneralizes all three. Autocompact uses a *forked agent* that inherits the parent's cached prefix, so only the summarization request is sent—a clean solution to the cost-of-compaction problem also not present in academic work.

4.2 MEMENTO

MEMENTO [4] (Microsoft Research, 2026) trains models to segment their own chain-of-thought into blocks, compress each into a memento, and physically remove the thinking block's KV entries mid-generation via a vLLM fork. It achieves 2–3× peak KV reduction and 1.75× throughput by enabling more concurrent requests in GPU memory.

MEMENTO and AdaptiveCacheare orthogonal: MEMENTO compresses within-step reasoning traces; AdaptiveCachemanages the across-step agentic context. Critically, MEMENTO *disables* prefix caching—its memento KV states are enriched with information from masked thinking blocks, violating the "same tokens ⇒ same KV values" assumption prefix caching requires. AdaptiveCache's layout optimization is the complement: it preserves the byte stability that MEMENTO deliberately sacrifices.

MEMENTO's key finding—that memento KV states implicitly encode information from masked blocks (removing this channel drops AIME24 accuracy by 15 pp)—suggests a novel scoring signal for AdaptiveCache: blocks heavily attended to by subsequent context have already propagated their information forward and are cheaper to evict.

5 Implementation

5.1 Two-Prototype Evolution

The implementation went through two distinct phases. The first was a lightweight Anthropic API prototype using

cache_control prompt-caching breakpoints, built to validate the core hypothesis quickly: does eviction hurt cache hit rate? Seven strategies were implemented and benchmarked on SWE-bench, confirming the hypothesis and producing the motivating results in §6. The second phase is the full system, introducing a GPU inference server (vLLM + LMCache), a KV block management API, and a three-tier eviction pipeline operating below the message level.

5.2 Core Library

The AdaptiveCachelibrary (src/adaptive_cache/) is infrastructure-independent and fully unit-tested (41 tests across 5 modules):

Module	Responsibility
segmenter.py	OpenAI messages → typed Block objects
scorer.py	2D importance×stability scoring
evictor.py	Budget cascade: SUFFIX→MIDDLE, never PIN
layout.py	Zone assignment; prefix-stable reordering
kv_controller.py	Three-tier controller; calls LMCache API

Block types, zone assignments, and scorer signals are validated with synthetic fixtures and hand-labeled examples. The library runs in CPU containers and communicates with the GPU server via Modal method calls.

5.3 GPU Inference Server

The server runs on a single A100-80GB GPU via Modal. It spawns vLLM as an OpenAI-compatible subprocess on port 8000 and waits up to 10 minutes for the health endpoint to respond. LMCache integrates as a LMCacheConnectorV1 plugin, which intercepts the KV transfer path. The internal management API is exposed on port 6999 via LMCACHE_INTERNAL_API_SERVER_ENABLED=True, providing /delete_blocks, /list_blocks, and /reset endpoints.

Measuring real prefix cache hit rates required polling vLLM's Prometheus /metrics endpoint and reading vllm:gpu_prefix_cache_hit_rate after each request. The num_cached_tokens field from the OpenAI-compatible API returned null in vLLM 0.8.5 (it is only populated in 0.9+), which required the indirect Prometheus approach.

5.4 Compatibility Engineering

Getting vLLM 0.8.5 and LMCache 0.4.3 to coexist required resolving three distinct issues.

LMCache build from source. The PyPI package for LMCache 0.4.3 did not include CUDA kernels for the A100's compute capability (sm_80). LMCache had to be built from source with TORCH_CUDA_ARCH_LIST='7.5;8.0;8.6' and SETUPTOOLS_SCM_PRETEND_VERSION=0.4.3 (to avoid git-based version detection failure inside Modal's build environment). This added significant image build time and required careful layer caching in the Modal image definition.

ABI mismatch (C++ shim). LMCache 0.4.3's CUDA bindings called c10::cuda::c10_cuda_check_implementation with an unsigned int line-number parameter. PyTorch 2.6.0 changed the signature to int. The symbol was resolved at load time but the ABI mismatch caused a segfault on the first CUDA error check. Fixed with a 10-line C++ shim compiled into a shared library and LD_PRELOAD'd at container startup:

```
namespace c10 { namespace cuda {
  void c10_cuda_check_implementation(
    int e, const char* f,
    const char* n, unsigned int l, bool b) {
    c10_cuda_check_implementation(
      e, f, n, static_cast<int>(l), b); }}}
```

vLLM 0.8.5 / LMCache 0.4.3 API mismatch. LMCache was written against vLLM 0.9+, which passes an engine_id string to the KV connector at construction. vLLM 0.8.5 does not. This caused an assertion error at startup. Separately, when the LMCache lookup client was None (fallback mode), the vLLM scheduler called a method that assumed it was populated, causing a second crash mid-request. Both were patched in LMCache source before installation:

```
# factory.py: supply default engine_id
metadata.engine_id = (
    metadata.engine_id or 'default-engine-0')

# vllm_v1_adapter.py: graceful degraded mode
if self.lookup_client is None:
    return 0, request_data
```

The patches are applied by patch_lmcache.py as part of the Modal image build, so the deployed container always has them without requiring a forked repository.

5.5 SWE-bench Harness

The harness runs mini-swe-agent with a custom AdaptiveCacheModel subclass that intercepts the message list before each API call. Four policies are implemented (none, fifo, adaptive, kv_adaptive). The harness runs 20 CPU containers in parallel (5 per policy), each cloning the relevant SWE-bench repository at the correct base commit and running the agent locally—avoiding Docker-in-Docker issues with Modal.

SWE-bench instance metadata (300 instances) is bundled as a local JSON file (swebench_instances.json), eliminating a runtime HuggingFace API dependency that caused flaky failures in early runs. Results are written to a Modal volume and aggregated by a local entrypoint after all containers complete.

5.6 Tier 1: The Unresolved Problem

Tier 1 block deletion requires identifying which LMCache CPU store entries correspond to evicted blocks. LMCache keys its store by a rolling SHA256 hash chain over 256-token chunks:

$$h_0 = \text{SHA256}(0_{\text{le64}} \,\|\, \text{chunk}_0)[:8]$$
$$h_i = \text{SHA256}(h_{i-1} \,\|\, \text{chunk}_i)[:8]$$

where token IDs are 4-byte little-endian. We re-implement this chain in `compute_block_hashes()` and POST target hashes to `/delete_blocks` on port 6999.

Despite the infrastructure being wired end-to-end, the call consistently returns `deleted=0`. The diagnostic tool `verify_block_hashes()` lists all keys in LMCache's CPU store and checks overlap with our computed hashes—but has not been successfully run in the deployed environment due to Modal's subprocess isolation (LMCache's engine runs inside the vLLM worker process, which is not directly accessible from the Modal method handler). All cache hit rate results below come from vLLM's built-in GPU prefix cache, not from LMCache block deletion.

The root cause of this isolation issue points toward a different implementation strategy for the final prototype: rather than calling an HTTP management API from outside the serving process, the right approach is to modify LMCache and vLLM directly—either via a fork (as MEMENTO did for its block masking) or by contributing the block-deletion interface upstream. This would eliminate the process-boundary problem entirely and give direct access to LMCache's internal state machine.

6 Results

6.1 Scope and Methodology

This project was primarily exploratory: build infrastructure, validate the core mechanism, identify what a rigorous evaluation requires. The implementation relied heavily on AI coding assistance (Claude Code), with the trade-off that systematic issues were documented but not always blocked on. A more carefully instrumented manual re-implementation is planned for the final prototype.

The vLLM SWE-bench experiments were attempted but all trajectories show zero steps completed—a container connectivity issue not investigated before the deadline. SWE-bench task results below are from the Anthropic API prototype. The vLLM stack produced one reliable result: the controlled cache hit rate experiment in §6.4.

6.2 Infrastructure Deployed

The full vLLM serving infrastructure was built and operated successfully. The harness dispatches up to 20 CPU agent containers in parallel (5 per policy across 4 policies) against a single A100-80GB GPU server running vLLM with LMCache. Each container clones the target repository locally, runs the agent, and writes results to a shared volume. The GPU server handled over 2,000 inference requests across all experiment runs, with vLLM's built-in prefix cache achieving a 76.3% server-side hit rate and saving an average of 10,977 cached tokens per request. This confirms the serving stack is functional at the required scale; the failure was in orchestrating the end-to-end SWE-bench pipeline reliably from within Modal containers, not in the inference server itself.

6.3 Anthropic API Prototype

The lightweight prototype used Anthropic's API with `cache_control` prompt-caching breakpoints. Seven strategies were compared on SWE-bench instances with Claude Haiku 4.5. Each strategy ran as an independent agent session, giving clean per-policy isolation.

Strategy	Cost	Turns	Hit rate
full (no eviction)	**$0.18**	**36**	75.8%
pairs	$0.26	40	67.4%
fifo	$0.43	55	65.8%
compaction	$0.62	51	49.6%

Table 3: Anthropic API prototype, astropy-12907, Claude Haiku 4.5. No-eviction is cheapest: eviction adds turns and cache misses that amplify cost rather than reduce it.

The result is counterintuitive: keeping full context is consistently cheapest and requires the fewest agent turns. FIFO costs 2.4× more despite evicting 30% of the context. Message-level eviction changes the byte sequence at the prefix, invalidating Anthropic's prompt cache—the savings on tokens are outweighed by the cost of recomputing previously cached content. Compaction is worst: the summary LLM call is expensive, the new summary tokens have no cached prefix, and the agent needs more turns to recover from lost context.

Over 5 instances, the pattern holds: a summarization-based strategy (episodes, not shown) achieves 85.5% cache hit rate but costs $0.31/instance vs full context at $0.23/instance, because it generates more total tokens via summarization calls.

6.4 Controlled vLLM Experiment

We ran a controlled 10-step coding conversation on the vLLM server (Qwen2.5-7B, budget=500 tokens, eviction forced at step 5).

Limitation: warm GPU cache. Policies ran sequentially on the same GPU server. `none` ran cold (step 1: 0% hit rate, 158s latency). `fifo` and `kv_adaptive` ran with vLLM's GPU cache already warm (step 1: 92.3% hit rate, $<$1.2s latency).

Policy	Mean hit rate	Post-eviction
none (full context)	94.4%	—
kv_adaptive	98.6%	99.2%
fifo	40.1%	**15.7%**

Table 4: vLLM prefix cache hit rate. Policies run sequentially; LMCache CPU store reset between runs, but GPU cache is not (see text).

This inflates their mean hit rates, making the mean comparison (98.6% vs 94.4%) unreliable.

What the data does show. The post-eviction divergence is not explained by warm cache. At step 5, FIFO drops to 15.7%: it removes the oldest messages, changing the prefix byte sequence—even warm cached tokens no longer match. kv_adaptive holds at 99.2%: it removes the *newest* messages, keeping the oldest content at the prefix unchanged. This confirms the core claim: evict from the suffix and the prefix cache survives; evict from the prefix and it collapses.

What is not tested. Tier 1 KV block deletion is broken (returns 0). All hits are from vLLM's GPU prefix cache, not LMCache. The experiment validates message-level suffix preservation, not the hole-leaving KV mechanism.

6.5 SWE-bench Task Results

The SWE-bench task experiments using the vLLM server produced zero completed trajectories due to the container connectivity issue described above. The following results are from the Anthropic API prototype (Claude Haiku 4.5, 5 astropy instances, 8K budget):

Policy	Submitted	Cost	Steps	Hit rate
none (unlimited)	5/5	$0.31	43	95.3%
fifo (8K)	5/5	$0.40	51	53.6%
adaptive (8K)	0/5	$0.17	30	61.1%

Table 5: SWE-bench task experiment, Anthropic API (Haiku 4.5), 5 astropy instances, 8K budget. Resolve rates unknown (`sb-cli` not run).

Adaptive submits no patches despite being cheapest ($0.17) and fastest (30 steps). The heuristic scorer evicts file reads the agent needs later, causing re-reads, extra turns, and eventual failure to produce a patch. FIFO costs 28% more than full context but at least retains the content the agent needs. These results confirm the scoring problem identified in §7: the mechanism is right, the scoring is not.

7 Reflection and Future Directions

7.1 The Scoring Problem

The controlled experiment validates the *mechanism*: suffix-first eviction preserves prefix cache hits. The SWE-bench result (0/5 patches from adaptive vs 5/5 from full context) exposes the *scoring problem*: the heuristic scorer does not know what to keep.

The current scorer uses structural type priors and reference counting—it cannot distinguish a file read the agent will need in 10 steps from one it glanced at once. Without this, the system evicts needed content, forces re-reads, and increases cost. MEM1 [14] and MemAgent [12] show the same failure mode: learned compression applied training-free consistently hurts. Importance estimation is empirical, not structural.

The near-term pivot. Collect 100+ SWE-bench agent traces with `-attention-backend EAGER`; derive oracle labels (did this block get attended to later? did evicting it correlate with failure?); train a lightweight scorer (MLP, pairwise ranking loss) on those labels; evaluate against the heuristic baseline at equal task-success. The current infrastructure is the right substrate. The heuristic system is the baseline to beat.

7.2 A Longer-Term Synthesis: Tetris + Folding + MEMENTO

Working through the related systems surfaced an insight that may be more valuable than the current design. We describe it as a future research direction.

The Tetris observation. When an agent works on a coding task, it progresses through a series of sub-problems: locate the bug, understand the affected files, make an edit attempt, run the tests, handle the failure. Each sub-problem generates a burst of context (error messages, file reads, bash outputs, edit attempts). When the sub-problem is *resolved*, that entire burst becomes dead weight. If those blocks were placed in the suffix, they can be evicted as holes at zero cost, leaving the prefix untouched.

The implication: if the rate of sub-problem completion roughly keeps pace with new work starting, the context size is bounded at $|\text{stable prefix}| + |\text{active sub-problem}|$, indefinitely. The agent runs on a self-cleaning sliding window. Tier 3 summarization may never fire.

The limitation of existing approaches. Context Folding [9] already tracks sub-trajectory completion and folds finished work. But folding generates summary tokens—a new byte sequence—that invalidates the prefix cache at every fold event. MEMENTO [4] demonstrates in-place KV block eviction at scale, but explicitly disables prefix caching because its KV states are enriched with masked content.

Neither system achieves: sub-trajectory-aware eviction *and* prefix cache preservation simultaneously.

The proposed synthesis.

1. **Sub-problem tracking (from Folding):** identify sub-problem boundaries in the agent's trajectory—either via learned signals (like FoldGRPO) or structural heuristics (a git commit, a passing test, a completed tool chain). Place the active sub-problem's context in the suffix.
2. **In-place KV eviction without summary tokens (from MEMENTO infrastructure):** when a sub-problem completes, use MEMENTO's vLLM fork to physically remove its KV blocks. No summary tokens are generated. The work product is in the repository; the conversation history of getting there is not needed.
3. **Prefix stability (from AdaptiveCache):** the foundational context (task, key files, cross-sub-problem references) stays at fixed prefix positions throughout. Because we evict only suffix blocks and generate no new tokens, the prefix byte sequence is never touched—prefix caching remains active, unlike MEMENTO.

What makes this different from both parents. Context Folding pays a cache miss at every fold; AdaptiveCacheavoids it by evicting without generating summary tokens. MEMENTO preserves implicit KV information from masked blocks; in the agentic setting this is unnecessary—we are evicting genuinely completed work, not reasoning we might still need. The synthesis gets the sub-trajectory awareness of Folding, the in-place KV eviction infrastructure of MEMENTO, and the prefix cache stability of AdaptiveCache—a combination none of the three achieves individually.

Open questions. Does the agent need to reference resolved sub-problems? (Probably rarely for focused coding tasks; more often for open-ended research tasks.) Can sub-problem boundaries be detected without training, or does this require a FoldGRPO-style RL signal? How does this compose with the learned scorer described above?

8 Next Steps and Milestones

Immediate. Fix Tier 1 (verify hash scheme, validate hole-leaving demo); run `sb-cli` to get resolve rates on existing patch files; sweep budgets {4K, 8K, 16K, 32K} to find the Pareto frontier.

Medium-term (100% milestone). Collect attention traces at scale (`-attention-backend EAGER`, 100+ instances); derive oracle importance labels; train and evaluate a learned scorer against the heuristic baseline.

125% milestone. GRPO-based reinforcement signal over eviction decisions, or MEMENTO-style agent self-management for agentic (tool-use) contexts.

9 Conclusion

The layout gap is real, and the mechanism for closing it works: suffix-first eviction preserves prefix cache hits; prefix-first eviction destroys them. What doesn't work is the heuristic scorer—confirmed by the SWE-bench experiment, by the Anthropic API prototype, and by the analogous failure mode in MEM1 and MemAgent. The Claude Code source confirms that surgical KV deletion without prefix invalidation is production-ready. MEMENTO confirms in-place masking at scale. The research direction pivots: from tuning heuristic importance weights to collecting attention traces and training a learned scorer grounded in what the model actually attends to. The current infrastructure is the apparatus for that experiment.

Milestone	Status
75%	*Substantially complete.* Core library (41 tests), baselines, harness, controlled experiment validating suffix eviction ⇒ prefix cache stability.
100%	*Redefined.* Not more heuristic signals—attention trace collection + learned scorer trained on oracle labels.
125%	*Redefined.* GRPO or MEMENTO-style agent self-management for tool-use contexts.

References

[1] All-Hands.dev. OpenHands context condensation. https://github.com/All-Hands-AI/OpenHands, 2025.

[2] Anthropic. Claude Code compaction source (`src/services/compact/`), 2025. Internal source, became publicly accessible 2026.

[3] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, et al. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

[4] Vasilis Kontonis, Yuchen Zeng, Shivam Garg, Lingjiao Chen, Hao Tang, Ziyan Wang, Ahmed Awadallah, Eric Horvitz, John Langford, and Dimitris Papailiopoulos. MEMENTO: Teaching LLMs to manage their own context. *arXiv preprint*, 2026. Microsoft Research.

[5] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[6] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12:157–173, 2024.

Phase	PIN prefix		SUFFIX
Sub-problem A active (steps 1–10)	`[sys][task][ast.py][utils.py]`		$[\texttt{err}_\texttt{A}][\texttt{bash}_\texttt{A}][\texttt{fix}_\texttt{A}]\ldots$
A resolves: in-place KV eviction, no summary	`[sys][task][ast.py][utils.py]`	*(unchanged)*	$[\cancel{\texttt{err}_\texttt{A}}][\cancel{\texttt{bash}_\texttt{A}}][\cancel{\texttt{fix}_\texttt{A}}] \leftarrow$ holes
Sub-problem B active (steps 11–20)	`[sys][task][ast.py][utils.py]`	*(unchanged)*	$[\texttt{err}_\texttt{B}][\texttt{bash}_\texttt{B}][\texttt{fix}_\texttt{B}]\ldots$

Figure 1: Proposed Tetris synthesis across two sub-problems. PIN prefix bytes are identical throughout all three phases—continuous KV cache hits. No summary tokens are generated at any point. Context size stays bounded at $|\text{prefix}| + |\text{current sub-problem}|$.

[7] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhuo Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. ScissorHands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] Yucheng Lu et al. Scaling LLM multi-turn reinforcement learning with end-to-end summarization-based context management. *arXiv preprint*, 2025.

[9] Yuxiao Sun et al. Context folding: Active context management for LLM agents via learned branch-return mechanisms. *arXiv preprint*, 2025.

[10] Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, et al. ReSum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*, 2025.

[11] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.

[12] Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiying Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, et al. MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent. *arXiv preprint arXiv:2507.02259*, 2025.

[13] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[14] Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*, 2025.